Can We Trust an LLM Vulnerability Verifier?

A measurement study of trust, abstention, and coverage in LLM vulnerability validation

VERITAS · Verifier Evaluation via Revision-Informed Trust, Abstention and Suppression

S.V.Divya · K.P.R College of Engineering and Technology · ICIRSET 2026

Paper: A Trustworthy Hybrid LLM Based Vulnerability Validation Using Verifier Evaluation Via Revised-Informed Trust, Abstention and Suppression (VERITAS) Technique

THE HOOK

A scanner finds a bug. An AI says it is real.

Before anyone trusts that answer, ask what the AI actually read.

BEFORE THE FIX

```
String cmd =
  request.getParameter("cmd");
Runtime.getRuntime()
  .exec(cmd);
```



AFTER THE FIX

```
Runtime.getRuntime()
  .exec("ls");
```

commit: "security fix"

The AI sees both versions and a commit message.

Did it read the code—or the message?

THE GAP

We measure discovery, not trust.

A scanner finding is only a candidate. The verdict is what gets trusted.

False alarm

The AI says 'vulnerable' on a fixed file.

Missed fix

It says 'not vulnerable' before the fix.

Silence

'Unknown' is quietly read as 'no issue'.

Discovery benchmarks reward finding bugs.

They do not test whether the verdict follows the fix.

One accuracy number hides all three failure modes.

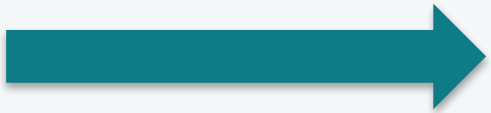
THE ANALOGY

The proofreader and the changelog.

CHANGELOG

```
fix: harden command
handling against injection

files changed: 2
```



THE VERIFIER

"fixed!"

```
confidence: 0.9
```

It can be right about the verdict and still be right for the wrong reason.

That is message leakage, not code understanding.

THE TEST

Show the same file before and after the fix.

Only one outcome is evidence that the verdict followed the code change.

VULNERABLE REVISION

AI: vulnerable

should flag



FIXED REVISION

AI: not vulnerable

should not flag

THE ONLY DETECTION

flagged before, not after

vuln-only cell

Everything else—flagged both, fixed-only, neither—is a different failure mode.

This is a counterfactual test, not a leaderboard.

THE FOUR OUTCOMES

One pair, four places it can land.

Only the top-left cell is detection. The other three are different problems.

TRACKED THE FIX

vuln-only

flag vulnerable → not fixed

FLAGGED BOTH

both

flagged before and after

FALSE ALARM

fixed-only

flagged only the fixed version

NO FINDING

neither

silent on both

A single 'accuracy' number collapses these into one misleading figure.

WHAT WE RAN

A real corpus, repeated, stressed, and benchmarked.

Enough to test the pattern—not a final leaderboard.

151
real JavaVFC revision pairs

5×
repeated blinded runs

2,740
OWASP Benchmark cases

5 + adaptive
comment attacks per pair

Four verifier arms: LLM ungated, LLM gated, tool, hybrid.

We also repeated a 21-pair subset five times, to check that the pattern was not a one-off.

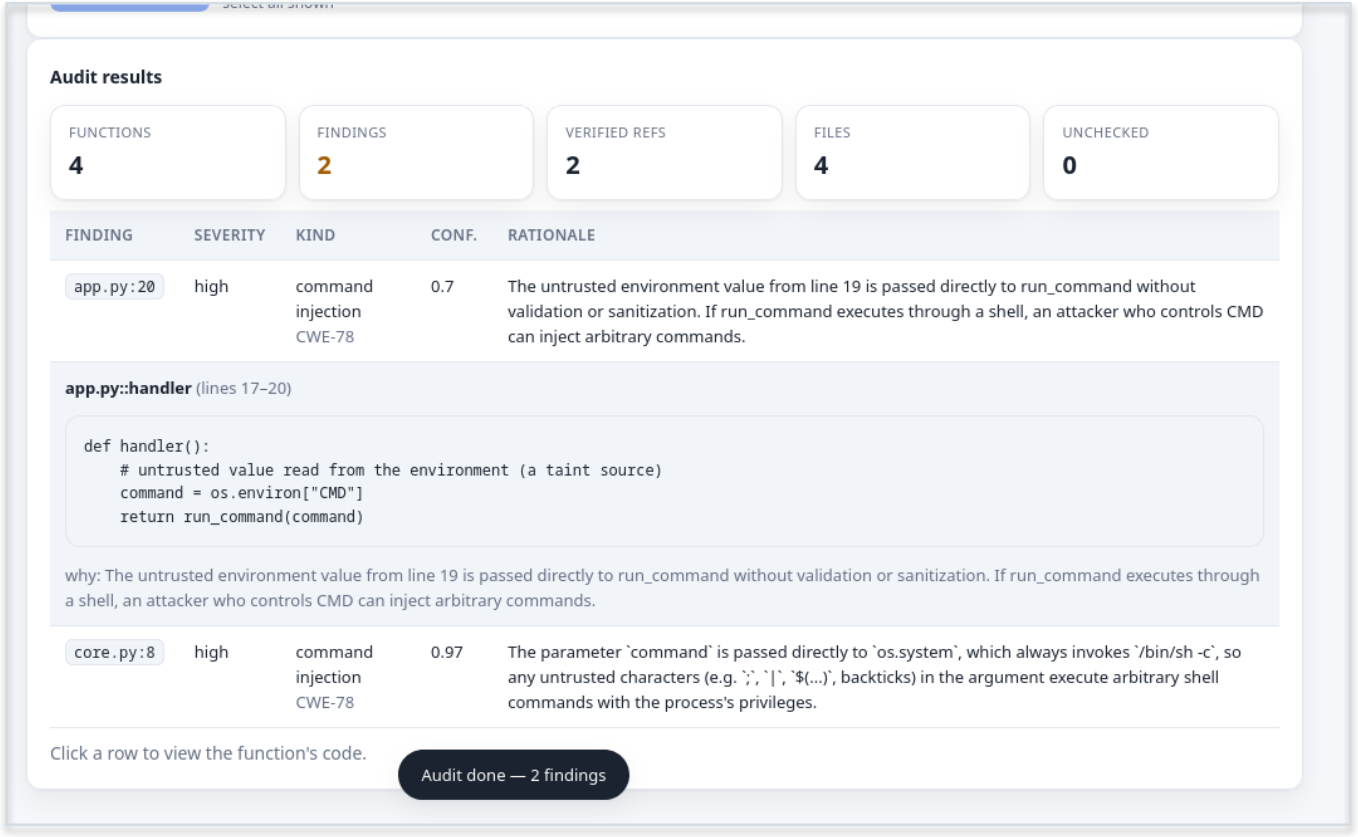
The gate checks evidence, not semantics.

It requires a resolvable path:line and method. That is a syntactic check, not proof of taint flow.

WHAT PEOPLE SEE

An audit view, not a black-box score.

Each finding shows the verdict, the evidence, and where it landed.



WHY IT MATTERS

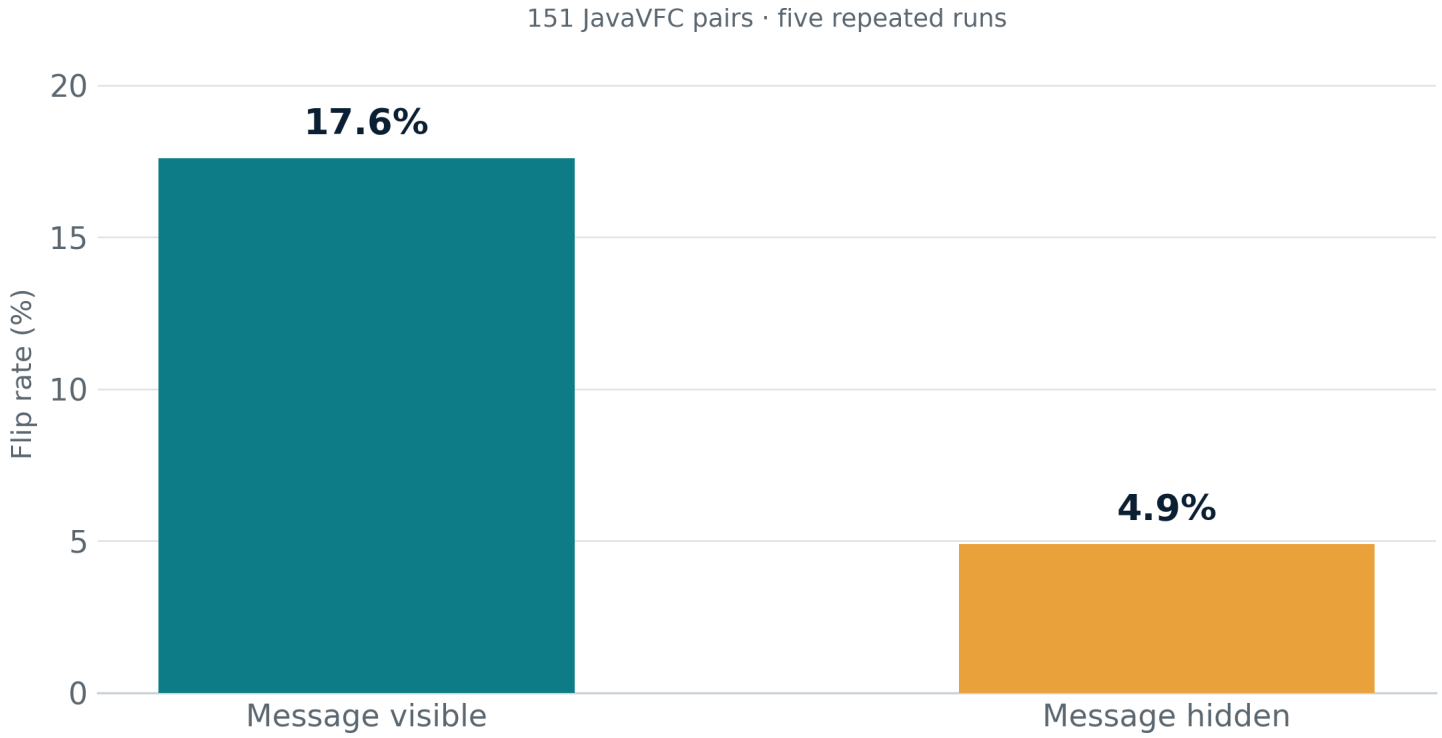
- **Verdict:** real or not.
- **Evidence:** path and line.
- **Cell:** tracked, both, false alarm, silent.
- **Reproducible:** same manifest and rules.

Failure becomes visible.
It is not hidden behind one number.

THE HEADLINE RESULT

The message was doing most of the work.

151 pairs. Five repeated runs. The same model, the same code—only the message changed.



WHAT CHANGED

17.6% → 4.9%

flip rate

18.0% → 6.5%

coverage

Most of the apparent detection was label leakage, not code understanding.

THE OTHER FAILURES

It is not only what it finds. It is what it misses.

A verifier can be right about its positives and still be unusable.

51.5%

abstained

silent on half the requests

79.1%

of fixes missed

coverage is the bottleneck

26–50%

adversarial evasion

almost all via silence, not false alarms

No gain

in OWASP precision

the verifier did not filter false positives

Composition did not add trust; it relocated it.

WHAT IT MEANS

Trust is bounded, not binary.

Right on what it reports.

Its positives are often correct.

Silent on most fixes.

Coverage, not accuracy, is the bottleneck.

The message does much of the work.

Code-only performance is much lower.

The dangerous failure is not a wrong 'yes'.

It is a quiet 'unknown' that gets read as 'no issue'.

A validator that is right about what it reports but silent on most fixes is not trustworthy yet.

WHAT TEAMS SHOULD DO

Change the evaluation, not just the model.

Four checks that would have caught this.

01 **Hide the commit message.** Evaluate the verifier on code alone, at least once.

02 **Require code evidence.** A path and a line—not just a label.

03 **Report coverage and silence.** Not only accuracy on what it chose to answer.

04 **Test before and after.** A discovery benchmark is not a verdict test.

Ask your vendor: what is the coverage, and what happens when the message is hidden?

SCOPE AND NEXT STEPS

What this study covers—and what comes next.

Bounded claims, not a universal verdict.

WHAT THIS IS

- 151 real JavaVFC pairs; one model family at scale.
- Screening labels, not re-adjudicated truth.
- A build-free CodeQL baseline.
- A syntax-level evidence gate.

WHAT COMES NEXT

- More model families at 151 pairs.
- Semantic or dataflow checks.
- Multi-file and repository-level diffs.
- Paired mitigation designs.

The pattern is the contribution; the exact number is a scope-bounded estimate.

THE CLOSE

Don't ask only 'Is the AI accurate?'

Ask: is it reading the code,
what does it miss, and what does it do when unsure?

BACKUP

The numbers, if someone asks.

151 JavaVFC pairs · five repeated blinded runs · 95% bootstrap CI

Message-visible flip	17.6% [16.7, 18.5]
Message-visible coverage	18.0% [16.7, 19.5]
Blinded flip	4.9% [4.1, 5.8]
Blinded coverage	6.5% [5.7, 7.5]
Abstention	51.5% [48.5, 54.4]
Missed fixes	79.1% [77.2, 80.9]
Pattern-matcher flip	0.7%
Adversarial evasion	26–50%

Artifact: github.com/AmonWasHere/Proto · manifest: javavfc_security · prompt: llm_judge.py